Computer Vision Engineer Assignment

-Prashant Bharti
(10/01/2026)

Document: Custom VLM Design for Industrial Quality Inspection

Problem Summary:

Task: Build an offline Vision–Language Model (VLM) for PCB inspection that accepts natural language queries.
Input: Natural language queries.
Output: And returns structured answers: {defect_type, bounding_box, confidence}
Availability: 50,000 labeled PCB images (with bounding boxes)
Constraint: Runs < 2 seconds inference and avoids hallucinations common in generic VLMs

---

(A) Model Selection:

I would choose the Base Architecture as : BLIP-2 + Custom Detection Head

Reason: **BLIP-2** is selected as the base VLM because of its **modular architecture** consisting of a vision encoder, a lightweight **Q-Former** for vision–language alignment, and a language model. Other factors are:

Factor	Reason
Architecture	Decoupled design: Vision Encoder + Q-Former + LLM
Fine-tuning flexibility	Can freeze LLM, tune vision + Q-Former efficiently
Inference speed	Much faster than LLaVA/Qwen-VL
Industrial deployability	Easier to compress, quantize
Licensing	Permissive for commercial use
Hallucination control	Q-Former gives structured grounding

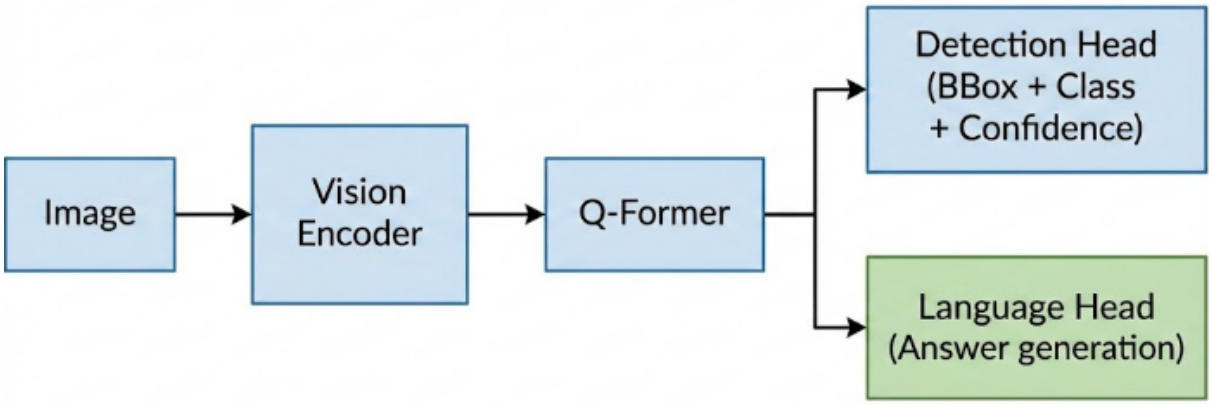
Reasons for not choosing LLaVA / Qwen-VL are:

1. Overly conversational
2. Large LLM dependency → slow

3. High hallucination risk
4. Harder to enforce structured output

Required Architectural Modifications

To enable **precise defect localization**, BLIP-2 is extended with a **dedicated detection head** attached after the Q-Former. The vision encoder is replaced with a **texture-sensitive backbone (ConvNeXt/ViT-S)**, and the Q-Former learns **defect-specific visual tokens**. A YOLO-style detection branch predicts **bounding boxes, defect class, and confidence**, while the language decoder is constrained to generate responses strictly grounded in detection outputs. This dual-head design ensures tight coupling between language answers and spatial evidence, eliminating vague descriptions and enabling accurate, verifiable localization.
Overview of the proposed PCB inspection VLM showing visual feature extraction, defect localization, and grounded language generation is shown in the figure:



(B) PCB-Specific Architecture Design

The VLM is designed as a detection-grounded system. A high-resolution vision encoder extracts fine PCB features and passes them to a Q-Former, which learns defect-focused visual tokens. These tokens are fused with the language model for question answering, while a parallel detection head predicts bounding boxes, defect type, and confidence. The language output is constrained to reference only detected defects, ensuring accurate, spatially grounded inspection results.

Full Architecture

[Query Image]
↓
[Vision Encoder (ConvNeXt-Tiny or ViT-S)]
↓
[Q-Former (learns defect tokens)]
↓
[Multi-Modal Fusion]

↓ ↓

[Detection Head] [Language Decoder]

[(BBox + Class)] [(Text response)]

Key Customizations: Following components may be modified as follows-

Component	Customization
Vision Encoder	Replace ViT-L with **ConvNeXt-Tiny** (fast, accurate on textures)
Q-Former	Train on defect-specific queries
Fusion	Cross-attention with defect tokens
Detection Head	YOLO-style head for precise localization
Output	Strict JSON schema output

An example output is:

```
{
 "defect": "missing_via",
 "location": [312, 418, 356, 455],
 "confidence": 0.93
}
```

(B) Optimization

To achieve real-time performance(<2s inference) on industrial hardware while preserving accuracy along with having offline deployment, the VLM is optimized at model, training, and deployment levels using the following techniques:

1. Model Size Reduction

1. **Lightweight Vision Encoder**
   Replace heavy backbones (ViT-L, ResNet-101) with **ConvNeXt-Tiny or ViT-S** to preserve fine-detail sensitivity while reducing computation.

2. **Compact Language Model**
   Use a small instruction-tuned LLM (1–3B parameters) rather than large generative models.

3. **Selective Fine-Tuning**
   Freeze the language model; fine-tune only the **Q-Former and detection head**, reducing both training and inference cost.

2. Quantization

1. Apply **INT8 quantization** to convolution, attention, and linear layers.
2. Reduces memory footprint by ~4×.
3. Improves inference speed by **2–3×** with minimal accuracy loss.

3. Structured Pruning

1. Remove redundant channels and attention heads based on magnitude and sensitivity analysis.
2. Typically prunes **30–40%** of the network with negligible performance drop.
3. Improves cache utilization and throughput.

4. Knowledge Distillation

1. Train a smaller **student model** using soft labels from a larger **teacher VLM**.
2. Retains most of the performance while dramatically reducing latency and memory usage.

5. Low-Rank Adaptation (LoRA)

1. Use LoRA for fine-tuning instead of updating full model weights.
2. Allows domain adaptation with minimal parameter overhead.
3. Keeps the deployment model small and stable.

6. Runtime Optimization

1. Export final model to **ONNX / TensorRT**.
2. Use layer fusion, kernel optimization, and hardware-specific acceleration.
3. Enables predictable, low-latency execution on industrial GPUs.

7. Pipeline-Level Optimization

1. Asynchronous execution between detection and language modules.
2. Batch visual processing for multi-board inspection scenarios.
3. Pre-allocate memory buffers to avoid runtime overhead.

(D) Hallucination Mitigation

To reduce hallucinations, the VLM is tuned to behave as a **grounded inspection system** rather than a free-form generative model. The core idea is to ensure that **every language output is explicitly supported by visual evidence** produced by the detection module. This is achieved by aligning language generation tightly with detection outputs, penalizing unsupported claims during training, and restricting the model's output space through architectural constraints. As a result, the model learns to either provide accurate, evidence-backed answers or explicitly state uncertainty when visual confidence is insufficient.

1. Training Strategies

1. **Grounded Supervision:** All answers are conditioned on detection results; language outputs must reference predicted bounding boxes and defect classes.
2. **Negative Sample Training:** Include clean boards, ambiguous images, occluded defects, and low-quality samples to teach the model when **not** to report defects.
3. **Refusal & Uncertainty Training:** Train the model to respond with "No defect detected" or "Insufficient confidence" when confidence falls below a threshold.
4. **Hard Example Mining:** Emphasize visually confusing samples where hallucinations are likely.

2. Loss Functions

The training objective combines detection accuracy with hallucination control:

$$L_{total} = L_{det} + L_{lang} + \lambda L_{ground}$$

Where:
• L_{det}: Bounding box + classification + confidence loss
• L_{lang}: Language generation loss
• L_{ground}: Penalty when language tokens are not supported by detected visual evidence
• λ: Weighting hyperparameter

This enforces that language output cannot diverge from what the vision module confirms.

3. Architectural Changes

1. Add a **dedicated detection head** whose outputs directly condition the language decoder.
2. Introduce **cross-attention gating** so the language model can only attend to verified defect tokens.
3. Enforce **constrained decoding** using a strict JSON grammar to prevent free-form hallucinated content.
4. Integrate **confidence-aware decoding**, allowing the model to suppress claims when visual certainty is low.

(E) Training Plan

The training process follows a **multi-stage curriculum** that progressively teaches the model to understand PCB images, localize defects, and answer inspection queries reliably. Since no QA pairs are initially available, the pipeline includes an automated **QA generation stage**, followed by robust **data augmentation** and strict **evaluation using task-specific metrics**. This staged design ensures stable convergence, strong grounding, and low hallucination behavior.

1. Multi-Stage Training Strategy

Stage 1: Vision & Detection Pretraining
Train the vision encoder and detection head using the 50,000 annotated PCB images.

Loss:

$$L_det = L_bbox + L_cls + L_conf$$

This stage builds strong localization and defect recognition ability.

Stage 2: Synthetic QA Pair Generation: Automatically generates large-scale QA pairs from existing annotations.

Example generation rules:

Annotation	Generated QA
Defect = short circuit at (x,y)	*"Where is the short circuit located?"*
Two missing vias	*"How many missing vias are visible?"*
No defect	*"Is this PCB defect-free?"*

This produces around 500k–1M training QA samples.

Stage 3: Vision–Language Alignment

Train the Q-Former and language decoder using generated QA data while freezing the vision backbone.

Loss:

$$L = L_{det} + L_{lang} + \lambda L_{ground}$$

Stage 4: Robustness & Hallucination Hardening

Add hard cases:
clean boards, occlusions, blur, extreme lighting, partial crops.

---

2. Data Augmentation Strategy

Augmentation improves robustness to industrial conditions as:

Image Augmentation

1. Rotation, scaling, flipping
2. Gaussian noise, blur

3. Contrast & brightness shifts
4. Random occlusion and cutout
5. Perspective distortions

Defect-Specific Augmentation

1. Synthetic defect insertion
2. Partial defect masking
3. Background clutter injection

Language Augmentation

1. Paraphrase questions
2. Vary technical vocabulary
3. Insert ambiguous queries for uncertainty learning

3. Evaluation Metrics & Formulas

Some evaluation metrics to be used are:

Intersection over Union (IoU):

$$IoU = \frac{Area(B_{pred} \cap B_{gt})}{Area(B_{pred} \cup B_{gt})}$$

Mean Average Precision (mAP):

$$mAP = \frac{1}{N} \sum_{i=1}^{N} AP_i$$

Counting Accuracy:

$$Accuracy_{count} = 1 - \frac{|C_{pred} - C_{gt}|}{C_{gt}}$$

Language Answer Accuracy (F1 Score):

$$F1 = \frac{2 \cdot Precision \cdot Recall}{Precision + Recall}$$

Hallucination Rate:

$$Hallucination\ Rate = \frac{Unsupported\ Responses}{Total\ Responses}$$

(F) Validation

The model is evaluated on three critical dimensions:
counting accuracy, **localization precision**, and **hallucination rate**, each of which directly affects trust in an industrial environment.

1. Counting Accuracy Validation

Why: Inspectors frequently ask questions like *"How many solder bridges are present?"*. A small counting error can lead to wrong decisions on board rejection or repair.

How: For each test image, compare the predicted number of defects with the ground-truth count.

$$Accuracy_{count} = 1 - \frac{|C_{pred} - C_{gt}|}{C_{gt}}$$

The final score is the average over all test samples and the metric penalizes both over-counting and under-counting and reflects real inspection reliability.

2. Localization Precision Validation

Why: Even if the defect type is correct, an inaccurate bounding box makes the result unusable for repair or further analysis.

How: Compute **Intersection over Union (IoU)** between predicted and ground-truth boxes:

$$IoU = \frac{Area(B_{pred} \cap B_{gt})}{Area(B_{pred} \cup B_{gt})}$$

A detection is considered correct if IoU ≥ 0.5.
Overall localization quality is measured using:

$$mAP = \frac{1}{N} \sum_{i=1}^{N} AP_i$$

This ensures both correct classification and precise spatial alignment.

3. Hallucination Rate Validation

Why: In industrial inspection, **false claims are more dangerous than missed detections**. The model must not report defects that are not visually supported.

How: Manually verify model responses against detection outputs and ground truth.

$$Hallucination\ Rate = \frac{Unsupported\ Responses}{Total\ Responses}$$

Unsupported responses include:

1. Reporting a defect where none exists
2. Giving incorrect location information
3. Making confident claims with low visual evidence

- - - —Thank You— - - -